Temat: Od Dockera do Kubernetesa · dzień 1: od kontenera do Docker Compose

Termin: 5 dni, 9:00–17:00

Prowadzi: Dawid Marniok

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Zaufaj naszemu doświadczeniu

18 lat
na rynku usług IT

500+
klientów biznesowych

29 560+
przeszkolonych osób

4 877+
zorganizowanych szkoleń i warsztatów

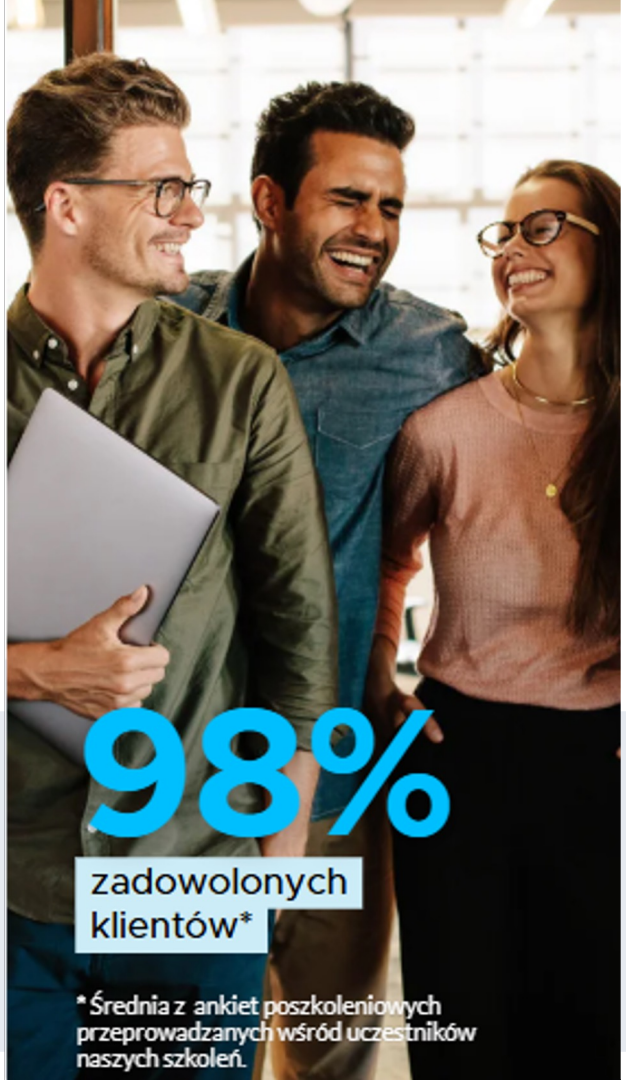
GWARANCJA JAKOŚCI USŁUG

Diamenty Miesięcznika Forbes 2023

ISO 9001:2015 CERTIFIED

STANDARD USŁUGI SZKOLENIOWEJ

Baza Usług Rozwojowych
uslugirozwojowe.parp.gov.pl



Edukacja na najwyższym poziomie

Wiedza specjalistyczna dla branży IT

Oferujemy szeroki katalog szkoleń z technologii mainstreamowych
i specjalistycznych, wschodzących i legacy. Zajęcia prowadzimy w trybie warsztatowym, a programy są oparte o praktyczne know-how. Specjalizujemy się w prowadzeniu dedykowanych szkoleń technologicznych, których agendę dostosowujemy do potrzeb naszych klientów i oczekiwań uczestników.

Wybitni eksperci

Od początku naszego istnienia przeszkoliliśmy dziesiątki tysięcy osób, co pomogło absolwentom podnieść konkurencyjność na rynku pracy i jakość projektów realizowanych na co dzień. Nasze szkolenia prowadzą najlepsi trenerzy, a nasze produkty są oparte na najnowocześniejszej technologii. Ich niezawodność i dopasowanie do potrzeb klientów są możliwe dzięki zespołowi składającemu się z wybitnych ekspertów i ekspertek, którzy/e są na pierwszej linii teorii i praktyki tworzenia i wdrażania innowacji technologicznych.

NASZE POZOSTAŁE MARKI EDUKACYJNE

STACJA.IT

kodoamacz by sages

sages

Najlepsze standardy usług edukacyjnych

Stosujemy Standard Usługi Szkoleniowej Polskiej Izby Firm Szkoleniowych, a nasze usługi realizowane są na najwyższym poziomie, o czym świadczą stale powracający klienci oraz wdrożony certyfikat ISO 9001. Metodologia prowadzonych przez nas zajęć oparta jest na współczesnych narzędziach i dostosowana do potrzeb i oczekiwań klientów.

Ponadto jesteśmy firmą wpisaną do rejestru instytucji szkoleniowych w Wojewódzkim Urzędzie Pracy w Warszawie pod nr 2.14/00133/2019.

Zaufali nam najlepsi

Wśród naszych klientów są takie firmy jak Alior Bank, OLX Group, Bank Zachodni WBK, Orange Polska, Lufthansa i wiele innych.

STUDIA PODYPLOMOWE

Wspieramy organizację zaawansowanych kierunków studiów podyplomowych. Realizujemy zajęcia na kierunkach: Data Science, Big Data i Wizualna analityka danych, AI & Data Driven Business oraz User Experience Design – projektowanie doświadczeń cyfrowych.

Instytut Informatyki
Wydział Elektroniki i Technik Informacyjnych Politechniki Warszawskiej

AKADEMIA LEONA KOŹMIŃSKIEGO

Sylwetka trenera

Dawid Marniok



Od lat pracuję jako DevOps i wdrażam DevSecOps: bezpieczeństwo jako część pipeline'u i klastra, a nie osobny etap na końcu.

Kontenery i Kubernetes wdrażam w projektach na Azure i AWS: od Dockerfile, przez Helm i CI/CD, po monitoring.

Zajmuję się AI for DevOps: jak AI i agenci pomagają w pracy z infrastrukturą, CI/CD i Kubernetesem.

Szkolę z Kubernetesa, chmury i narzędzi DevOps. Doradzam firmom przy wyborze i wdrażaniu narzędzi DevOps.

LinkedIn:
linkedin.com/in/dawidmarniok

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Organizacja: pięć dni, te same zasady

Czas	9:00–17:00 · przerwy 10:45–11:00, obiad 13:00–13:45, 15:00–15:15
Pomoc	problem w labie → czat albo priv · udostępnisz ekran
Zoom	mikrofon wyciszony · pytanie: ręka albo czat, w każdej chwili
Repo	github.com/dawid-marniok/kubernetes-w-praktyce · rano git pull
Dziś	Docker Desktop na laptopie · od jutra klaster Kubernetes
Zasada	każdy lab kończy się czymś, co działa u Ciebie

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

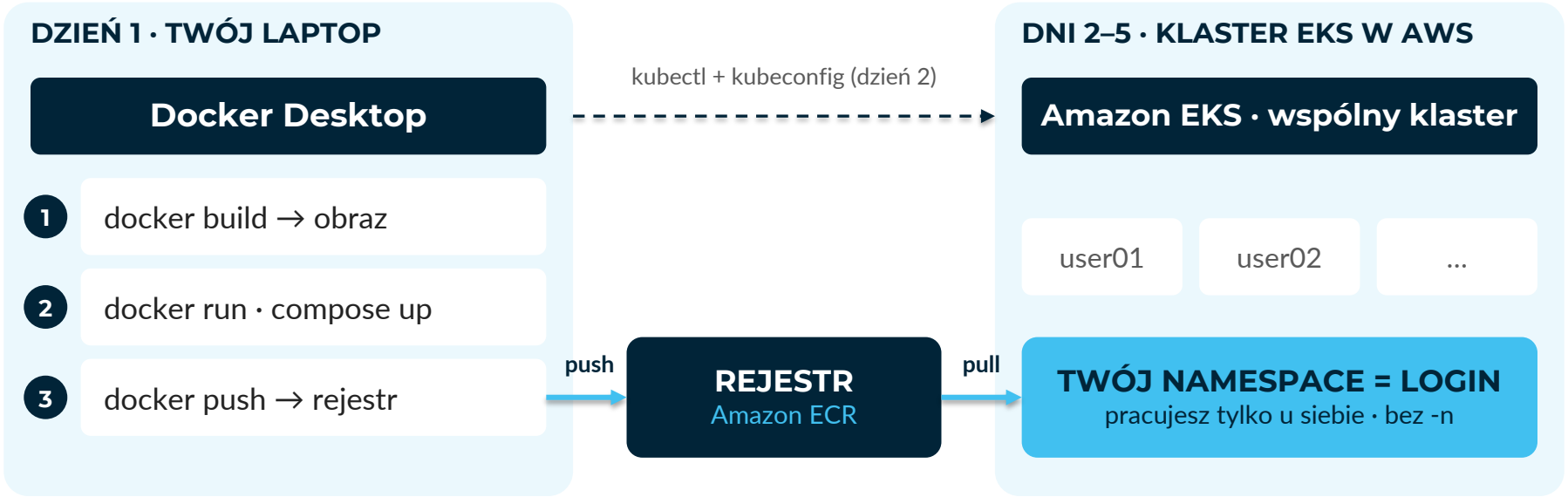
Pięć dni: Docker, potem cztery dni Kubernetesa

DZIEŃ 1 · DOCKER	DZIEŃ 2 · K8S	DZIEŃ 3 · K8S	DZIEŃ 4 · K8S	DZIEŃ 5 · K8S
Kontenery i podstawy	Architektura i kubectl	Storage i StatefulSet	RBAC i ServiceAccount	Helm
Sieci i dane	Pod i Namespace	DaemonSet, Job, CronJob	Pod Security i admission	Operatory
Obrazy i Dockerfile	Deployment i konfiguracja	Zasoby, QoS, autoskalowanie	Sieć: CNI, NetworkPolicy, DNS	Monitoring i observability
Bezpieczeństwo i Compose	Service, Ingress, Gateway API	Planowanie podów i utrzymanie	Rozszerzanie API	Projekt końcowy ToDos

dzień 1: ToDos w Compose · dni 2–5: ten sam ToDos rośnie na klastrze, dzień po dniu

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Gdzie pracujesz: dziś laptop, od jutra klaster EKS



obraz zbudowany dziś na laptopie uruchomisz jutro w swoim namespace w EKS

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

TERAZ · Na start

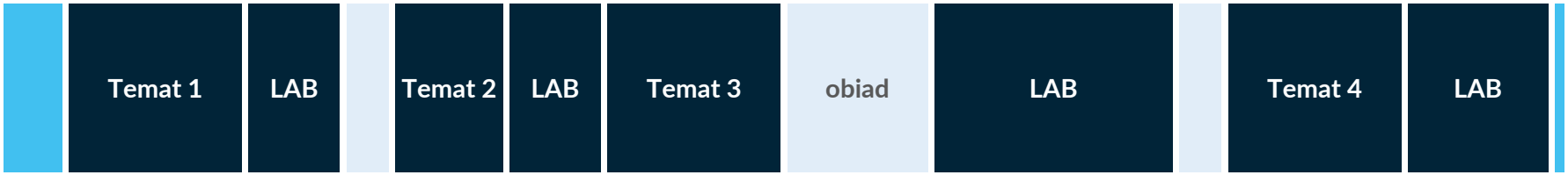
Trzy komendy, zanim zaczniemy

```
1. docker version                → Client i Server, oba z numerem wersji
2. docker run --rm hello-world → „Hello from Docker!”
3. git clone https://github.com/dawid-marniok/kubernetes-w-praktyce
   cd kubernetes-w-praktyce
   ls                            → app/  labs/  przyklady/  setup/ …
```

NA CZAT wersja Server z docker version · albo błąd, na którym stoisz

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Dzień 1: od pierwszego kontenera do Compose



9:00 10:45 13:00 15:00 17:00

1. Kontenery i podstawy
2. Sieci i dane
3. Obrazy i Dockerfile
4. Bezpieczeństwo i Compose

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

1

DZIEŃ 1 · TEMAT 1

Kontenery i podstawy

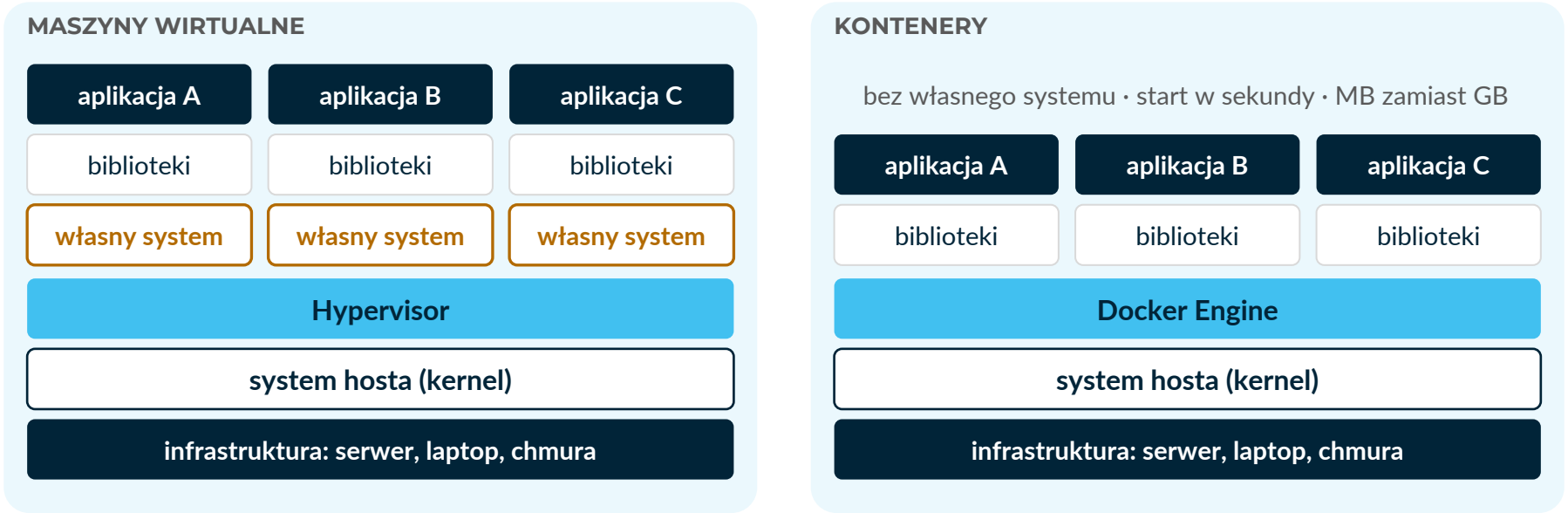
Po tym temacie wiesz, czym kontener różni się od VM, i uruchamiasz go z portem, zmiennymi i wolumenem.

DOCKER	DZIEŃ 2	DZIEŃ 3	DZIEŃ 4	DZIEŃ 5
1 2 3 4	1 2 3 4	1 2 3 4	1 2 3 4	1 2 3 4

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Kontener nie ma własnego systemu



kontener to proces na wspólnym kernelu hosta, nie mała maszyna wirtualna

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

12-factor: 12 zasad aplikacji gotowej na kontener

I Codebase
jedno repozytorium

II Dependencies
zależności

III Config
konfiguracja

IV Backing services
usługi zewnętrzne

V Build, release, run
budowanie, wydanie, uruchomienie

VI Processes
bezstanowość

VII Port binding
własny port

VIII Concurrency
skalowanie w poziomie

IX Disposability
szybki start i stop

X Dev/prod parity
te same środowiska

XI Logs
logi na stdout

XII Admin processes
zadania jednorazowe

niebieskie pola: zasady, bez których aplikacja źle działa w kontenerze i w Kubernetesie

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

CNCF: poziom mówi o dojrzałości, nie o jakości

Graduated szeroko na produkcji, audyt bezpieczeństwa	Kubernetes · containerd · Prometheus · Helm · Argo · KEDA · OpenTelemetry
Incubating rośnie, są wdrożenia produkcyjne	Backstage · OpenFeature · Keycloak
Sandbox wczesna faza, eksperymenty	k8sgpt · CloudNativePG · setki młodych projektów

Cloud Native Computing Foundation (Linux Foundation) · cncf.io/projects

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

POKAZ · landscape.cncf.io

Tysiąc narzędzi, kilka do zapamiętania

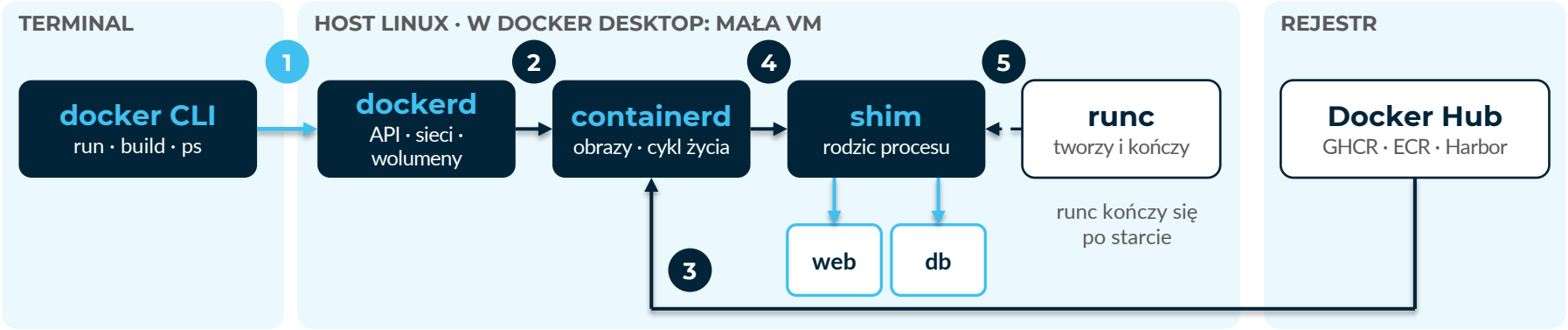
Cel: znaleźć na mapie projekty, z których korzystamy w tym tygodniu, i ich poziom.

OBSERWUJCIE

1. filtr Graduated
2. gdzie są Kubernetes i containerd
3. karta projektu z poziomem

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Docker to kilka procesów, nie jeden program



CLI	wysyła polecenia do dockerd przez API (docker.sock)
dockerd	API, sieci, wolumeny i build obrazów
containerd	pobiera obrazy, prowadzi cykl życia kontenera
shim	pilnuje kontenera: zbiera logi, kontener przeżyje restart Dockera
runc	tworzy namespaces i cgroups, startuje proces i znika
Kubernetes	kubelet → CRI → containerd, bez dockerd i CLI

docker run = pull obrazu + utworzenie kontenera + start procesu

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Desktop na laptopie, Engine na serwerze

	Gdzie	Co dostajesz	Licencja
Docker Desktop	Windows (WSL 2), macOS, Linux	VM z Linuksem, Engine, CLI, Compose, buildx, GUI	płatna w firmie > 250 osób lub > 10 mln USD
Docker Engine	Linux: serwer, CI	dockerd, containerd i CLI z pakietów	open source
Alternatywy	laptop	Podman Desktop, Rancher Desktop, OrbStack, Colima	zależnie od narzędzia

na szkoleniu: Docker Desktop • wbudowany Kubernetes zostaw wyłączony, jutro mamy klaster

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Codzienna praca to sześć komend

```
docker run -d --name web \
  -p 8080:80 nginx:1.29
docker ps -a
docker logs -f web
docker exec -it web sh
docker inspect web
docker stop web && docker rm web
```

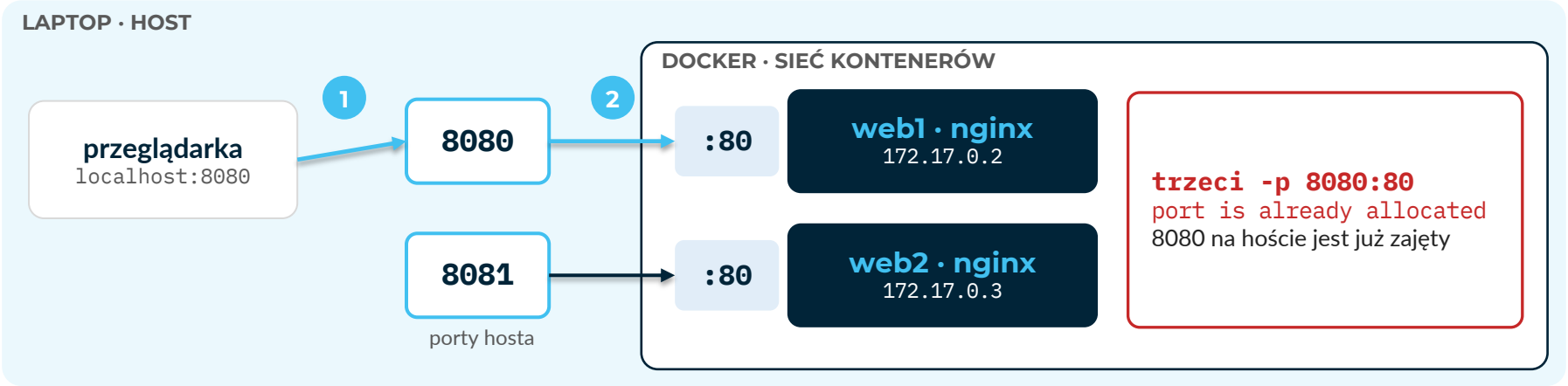
FLAGI RUN

```
-d      w tle
-it     interaktywnie
--rm    usuń po wyjściu
--name  własna nazwa
-p      host:kontener
-e      zmienna
```

zawsze tag obrazu: nginx:1.29, nie nginx · bez tagu dostajesz :latest

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

-p host:kontener: jeden port hosta, jeden kontener




```
docker run -d --name web1 -p 8080:80 nginx:1.29
docker run -d --name web2 -p 8081:80 nginx:1.29
```

każdy kontener ma własne :80 · port 8080 na hoście może zająć tylko jeden

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Konfiguracja wchodzi zmiennymi środowiskowymi

SKĄD

```
docker run -d --name db \
  -e POSTGRES_PASSWORD=changeme \
  -e POSTGRES_DB=shop postgres:18

docker run --env-file app.env …
```

GDZIE WIDAĆ

```
docker exec db env

docker inspect db \
  --format '{{.Config.Env}}'
```

ten sam obraz → **dev: -e LOG_LEVEL=debug** → **prod: -e LOG_LEVEL=warn**

nowa wartość = nowy kontener (docker rm + run) · hasło w -e widać w inspect i historii shella

12-factor III: konfiguracja poza obrazem · jutro to samo robi ConfigMap i Secret

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Kontener usunięty, dane zostały




```
docker volume create dane    docker run -v dane:/data …    docker volume ls
```

docker rm nie usuwa wolumenu · bind mount, tmpfs i backup w temacie 2

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Kontener żyje tyle, co jego główny proces

POLECENIE	CO ROBI GŁÓWNY PROCES	STATUS
`docker run -d nginx:1.29`	obsługuje ruch i nie kończy się	Up
`docker run ubuntu:24.04`	bash bez terminala nie ma nic do roboty, wychodzi	Exited (0) po 1 s
`docker run ubuntu:24.04 sleep 3600`	śpi godzinę, potem się kończy	Up przez godzinę

Pułapka: usługa uruchomiona „w tle” (skrypt z & na końcu, nginx bez daemon off)
skrypt startowy kończy się po sekundzie, a razem z nim kontener

kontener to nie mały serwer, tylko jeden program: skończy się program, kończy się kontener

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

▶ DEMO · Pierwszy kontener

Od Docker Desktop do terminala w osiem minut

Cel: port, zmienna i wolumen w Docker Desktop, potem w CLI.

OBSERWUJCIE

1. Containers, Images, Volumes
2. PORTS i Exited w docker ps
3. plik przeżywa docker rm

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

LAB `labs/lab01-docker-podstawy/`

Pierwsze kontenery na Twoim laptopie

1. hello-world, ubuntu interaktywnie, nginx w tle: logs, exec, inspect
2. Porty: drugi kontener na 8080, błąd i poprawka na 8081
3. Zmienne: postgres:18 z --env-file db.env, env i inspect
4. Wolumen: test.html przeżywa docker rm · chętni: stats, tagi

SUKCES localhost:8080 odpowiada, postgres działa ze zmiennymi z pliku, a test.html przeżył docker rm

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Co zostaje po docker rm, a co po docker rmi?

Następny temat: Sieci i dane

sieć użytkownika i DNS po nazwie, wolumen, bind mount i tmpfs, logi kontenera

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

2

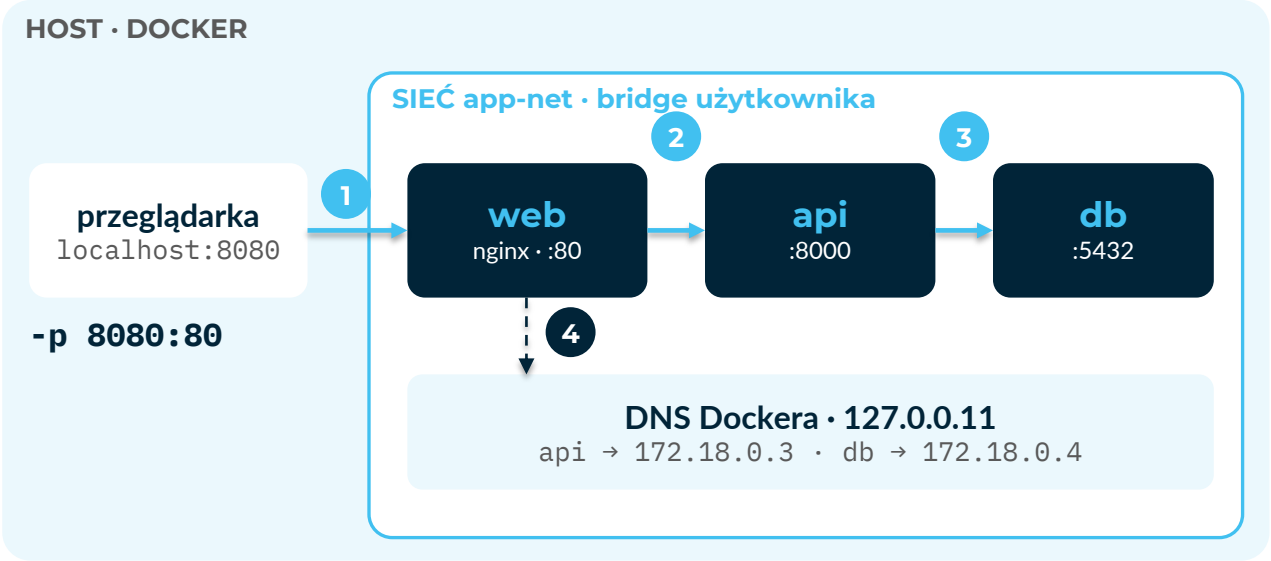
DZIEŃ 1 · TEMAT 2

Sieci i dane

Po tym temacie łączysz kontenery po nazwie, wybierasz, gdzie trzymać dane, i wiesz, skąd biorą się logi.

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

W sieci użytkownika działa DNS po nazwie



TYPY SIECI

- **bridge** — domyślna, bez DNS po nazwie
- **bridge użytkownika** — DNS po nazwie, izolacja
- **host** — stos sieci hosta, bez -p
- **none** — brak sieci
- **overlay** — wiele hostów, Swarm

-p otwiera port na hoście · między kontenerami wystarczy wspólna sieć i nazwa

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Dane poza kontenerem: wolumen, bind, tmpfs

Wolumen

zarządza nim Docker · przeżywa docker rm · bazy danych

```
-v pgdata:
  /var/lib/postgresql
```

Bind mount

katalog z hosta · kod w dev, pliki konfiguracji

```
-v ./html:
  /usr/share/nginx/html:ro
```

tmpfs

w pamięci RAM · znika przy stop · pliki tymczasowe

```
--tmpfs /tmp
```

bind mount zasłania pliki z obrazu: pusty katalog na hoście = pusty katalog w kontenerze

docker rm nie usuwa wolumenu · docker volume rm i docker compose down -v już tak

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Wolumen to trwałość, nie kopia zapasowa

wolumen pgdata → kontener alpine z tar → bind ./backup → pgdata.tar.gz na hoście

```
docker run --rm -v pgdata:/dane:ro -v ./backup:/backup alpine:3.22 \
  tar czf /backup/pgdata.tar.gz -C /dane .
```

Przeżywa

docker rm, restart Dockera, nowy obraz bazy

Nie przeżywa

docker volume rm, down -v, awaria dysku

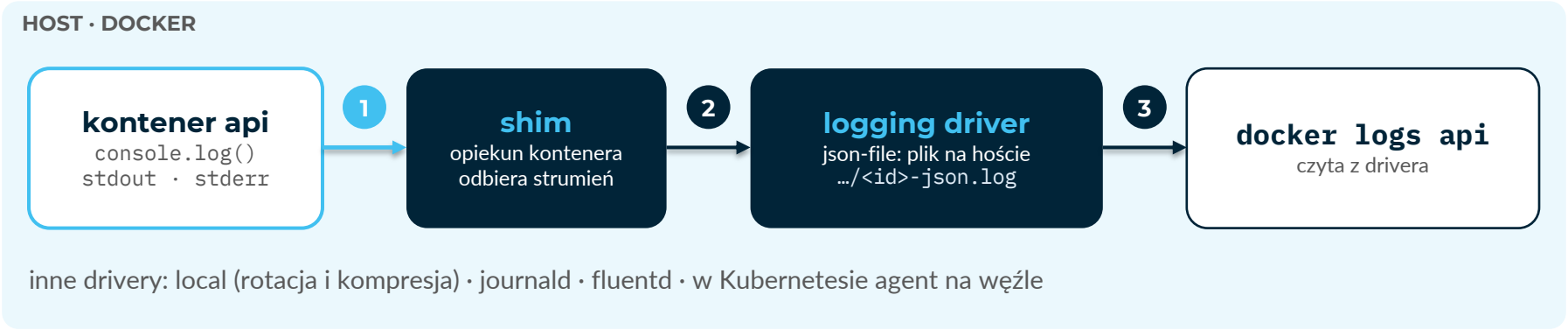
Baza danych

spójny backup: pg_dump, mongodump

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Logi to strumień, nie plik w kontenerze



log do /app/logs/app.log
docker logs pusty · plik znika przy rm · zapycha warstwę zapisu

```
docker run --log-opt max-size=10m \
  --log-opt max-file=3 …
```

json-file domyślnie nie rotuje: bez max-size gadatliwy kontener zapcha dysk

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

▶ DEMO · Sieć, dane, logi

Nazwa działa tylko w sieci użytkownika

Cel: domyślna sieć kontra app-net, wolumen kontra bind mount, plik logów na hoście.

OBSERWUJCIE

1. kiedy działa ping po nazwie
2. co przykrywa bind mount
3. gdzie leży plik logów

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

LAB `labs/lab02-sieci-wolumeny/`

Sieci i dane, które przeżyją kontener

1. Sieci: domyślna bez nazw, app-net z DNS, izolacja i network connect
2. Bind mount, tmpfs i wolumen: edycja na żywo, co znika po restarcie
3. Logi: --tail, --since, rotacja max-size, LogPath
4. Chętni: postgres na wolumenie, backup tarem, netshoot, podsieć

SUKCES nazwa działa w jednej sieci, a między sieciami nie, zmiana pliku widoczna od razu, a logi są przycięte

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Która z tych rzeczy trafi jutro do Kubernetesa?

Następny temat: Obrazy i Dockerfile

warstwy i cache, Dockerfile krok po kroku, multi-stage, sekrety w buildzie, rejestry i inna architektura

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

3

DZIEŃ 1 · TEMAT 3

Obrazy i Dockerfile

Po tym temacie budujesz mały, szybki obraz i wypychasz go do rejestru na właściwą architekturę.

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Obraz to stos warstw tylko do odczytu

DOCKERFILE	OBRAZ · WARSTWY	PO ZMIANIE W KODZIE
`FROM python:3.13-slim`	system i Python z obrazu bazowego	CACHED
`WORKDIR /app`	pusty katalog	CACHED
`COPY requirements.txt .`	lista zależności	CACHED
`RUN pip install -r …`	zależności, największa warstwa	CACHED
`COPY . .`	kod aplikacji	budowana od nowa

każdy kontener dokłada cienką warstwę zapisu

RW kontenera A | RW kontenera B

warstwy obrazu wspólne

zmiana warstwy przebudowuje ją i wszystkie warstwy nad nią

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Dockerfile: jedna instrukcja, jedna decyzja

```
FROM python:3.13-slim
LABEL org.opencontainers.image.version="1.0.0"
ARG APP_VERSION=dev
ENV PYTHONUNBUFFERED=1 APP_VERSION=$APP_VERSION
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN useradd -r -u 10001 app
USER 10001
EXPOSE 8000
HEALTHCHECK CMD ["python", "healthcheck.py"]
CMD ["python", "app.py"]
```

CMD vs ENTRYPOINT

CMD: domyślne polecenie, łatwo nadpisać · ENTRYPOINT: stały program, CMD to jego argumenty

COPY vs ADD

zawsze COPY · ADD tylko do rozpakowania tar albo pobrania z --checksum

ARG vs ENV

ARG istnieje tylko w buildzie · ENV zostaje w obrazie i w kontenerze

LABEL, nie MAINTAINER

MAINTAINER jest przestarzały · klucze org.opencontainers.image.*

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Zła kolejność wyłącza cache przy każdej zmianie

ŹLE

```
COPY . .
RUN pip install -r requirements.txt
```

DOBRZE

```
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
```

.dockerignore · obok Dockerfile

```
.git
node_modules
**/__pycache__
bin/  obj/
.env  *.pem
```

bez .dockerignore
COPY . . zabiera .git, lokalne node_modules i plik .env z hasłami

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Multi-stage: buduj w SDK, uruchamiaj w runtime

```
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src
COPY *.csproj .
RUN dotnet restore
COPY . .
RUN dotnet publish -c Release -o /app/publish

FROM mcr.microsoft.com/dotnet/aspnet:10.0
WORKDIR /app
COPY --from=build /app/publish .
USER $APP_UID
EXPOSE 8080
ENTRYPOINT ["dotnet", "sample.dll"]
```

ETAP 1 · build

SDK, kompilator, NuGet

zostaje w cache buildera, nie trafia do obrazu

ETAP 2 · obraz końcowy

tylko runtime i publish

mniej MB, mniej podatności, użytkownik app zamiast root

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Sekret w ARG lub ENV zostaje w obrazie

ŹLE

```
ARG NPM_TOKEN
ENV NPM_TOKEN=$NPM_TOKEN
RUN npm ci
```

docker history i docker inspect
pokażą token każdemu z obrazem

DOBRZE

1. token leży w pliku na laptopie albo w CI: ~/.npmrc
2. Dockerfile: podłącz sekret na czas jednego RUN

```
RUN --mount=type=secret,id=npmrc,target=/root/.npmrc \
    npm ci
```

3. build: wskaż plik, id łączy oba miejsca

```
docker build --secret id=npmrc,src=$HOME/.npmrc .
```

po RUN pliku nie ma: ani w warstwie, ani w historii obrazu
tak podajesz sekrety buildu (npm, NuGet) · hasło do bazy przychodzi przy uruchomieniu

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Pełna nazwa obrazu mówi, skąd go pobrać

`ghcr.io`	`acme`	`todos-api`	`:1.4.2`	`@sha256:…`
rejestr	przestrzeń	repozytorium	tag, ruchomy	digest, stały

PUBLICZNE	CHMUROWE	SELF-HOSTED
Docker Hub domyślny · obrazy oficjalne · limity pulli	**Amazon ECR ← nasz** AWS · EKS pobiera przez rolę IAM	**Harbor** CNCF · skanowanie, podpisy, replikacja
GHCR przy repo GitHub · CI w Actions	**Azure ACR** Azure · pull z AKS bez haseł	**registry:3** minimalny · lokalnie, testy, CI
Quay.io Red Hat · publiczne i prywatne	**Artifact Registry** Google Cloud · GKE	**Nexus · Artifactory** obrazy obok paczek npm, Maven

docker tag → docker login → docker push · bez nazwy rejestru push idzie do Docker Hub

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Obraz z Maca M nie wstanie na EKS amd64

Mac M: arm64 → docker build + push → EKS: węzeł amd64 → exec format error

```
docker buildx build --platform linux/amd64,linux/arm64 \
  -t <rejestr>/todos-api:1.0.0 --push .
```

```
docker buildx imagetools inspect <rejestr>/todos-api:1.0.0
```

jeden tag, dwie architektury: węzeł sam pobiera swój wariant

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

▶ DEMO · Build, cache, multi-stage

Drugi build trwa sekundy, obraz jest mniejszy

Cel: zwykły build nginx z cache, potem .NET jednoetapowo i w multi-stage.

OBSERWUJCIE

1. które kroki są CACHED
2. rozmiar w docker images
3. co pokazuje history

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

LAB `labs/lab03-dockerfile/`

Własny obraz, od Dockerfile do rejestru

1. Strona w nginx: COPY, LABEL i docker history
2. Node: kolejność warstw, npm ci, .dockerignore, USER node
3. Multi-stage .NET z przyklady/: porównaj rozmiary sample
4. Tag 1.0.0 i push do rejestru · chętni: Go, sekrety, healthcheck

SUKCES node-app działa jako uid 1000 i korzysta z cache, a node-app:1.0.0 jest w rejestrze

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Wasz obraz ma 1 GB. Od czego zaczniecie odchudzanie?

Następny temat: Bezpieczeństwo i Compose

bezpieczny obraz i docker run, docker.sock, potem cała aplikacja w jednym compose.yaml

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

DZIEŃ 1 · TEMAT 4

4 Bezpieczeństwo i Compose

Po tym temacie uruchamiasz kontener z minimum uprawnień i opisujesz aplikację w compose.yaml.

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Bezpieczny obraz: mało w środku, nie root

Nie root

USER 10001 w Dockerfile · obrazy node, .NET i distroless mają gotowego użytkownika

Minimalna baza

alpine, slim, distroless · Chainguard · Docker Hardened Images

Skanowanie

trivy image todos-api:1.0.0 · docker scout cves · w CI przy każdym buildzie

CVE w obrazie bazowym naprawia nowy build, nie restart kontenera

mniej pakietów w obrazie = mniej podatności do łatania i mniej narzędzi dla atakującego

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Kontener dostaje tylko to, czego potrzebuje

```
docker run -d --name api \
  --user 10001 --read-only --tmpfs /tmp \
  --cap-drop ALL --security-opt no-new-privileges \
  --memory 256m --cpus 0.5 --pids-limit 100 \
  -p 8000:8000 todos-api:1.0.0
```

`--read-only`
system plików tylko do odczytu, zapis w tmpfs i wolumenach

`--cap-drop ALL`
bez uprawnień roota w kernelu, dodaj tylko potrzebne

`--memory · --cpus`
limit, a po przekroczeniu pamięci OOM kill (137)

jutro to samo w Kubernetesie: securityContext i resources w manifeście poda

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

docker.sock w kontenerze to root na hoście

kontener ma docker.sock
czyli pilota do Dockera

→

każe Dockerowi uruchomić
kontener z dyskiem hosta

→

jest rootem na hoście
izolacji już nie ma

SZUKAJ W COMPOSE.YAML I W CI

```
- /var/run/docker.sock:/var/run/docker.sock
privileged: true
```

--privileged daje ten sam efekt inną drogą: kontener widzi dyski i urządzenia hosta

aplikacja nie potrzebuje ani docker.sock, ani --privileged

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

▶ DEMO · Bezpieczny kontener

Mniej w obrazie, mniej uprawnień przy starcie

Cel: skan dwóch obrazów nginx, potem --read-only, --cap-drop i bez roota.

OBSERWUJCIE

1. ile CVE ma każdy obraz
2. co psuje --read-only
3. kto jest UID w kontenerze

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Docker Compose

cała aplikacja w jednym pliku: usługi, sieć i wolumeny

sages

Pięć komend z ręki to nie jest konfiguracja

```
docker network create app-net
docker volume create pgdata
docker run -d --name db --network app-net \
  -v pgdata:/var/lib/postgresql \
  -e POSTGRES_PASSWORD=postgres postgres:18
docker build -t shop-web ./app
docker run -d --name web --network app-net \
  -p 5001:5000 shop-web
```

→

```
docker compose
up -d
```

- plik w repo obok kodu
- ta sama kolejność za każdym razem
- down sprząta wszystko
- review jak każdy kod

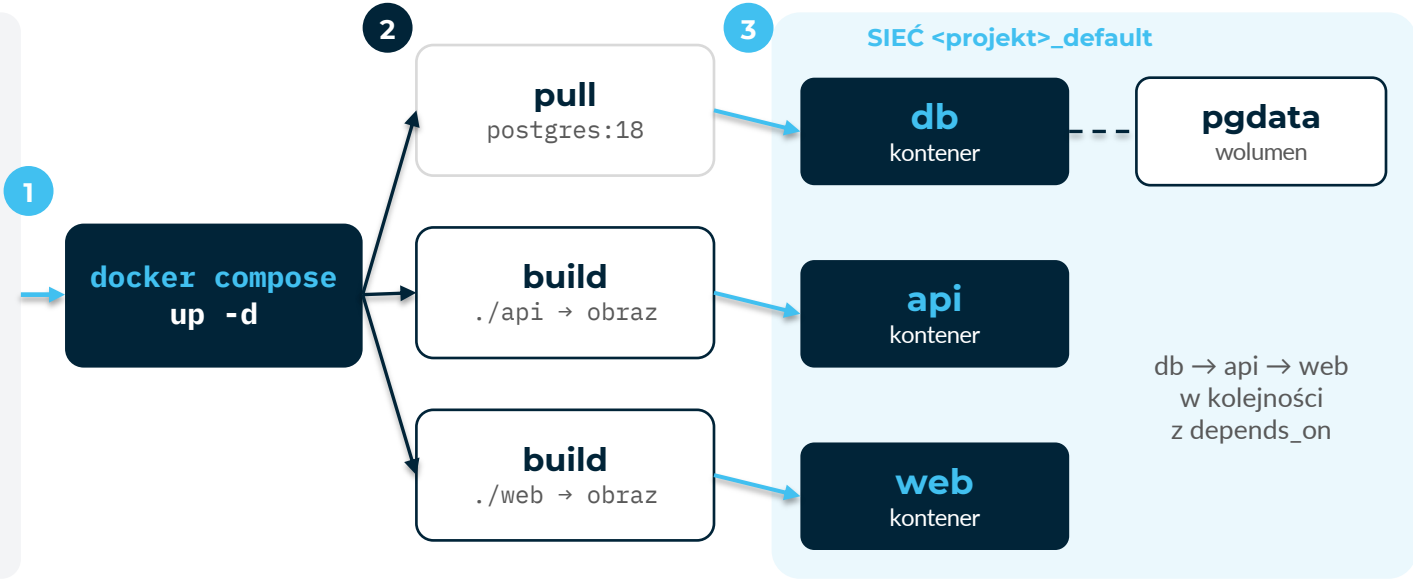
Compose = deklaracja: piszesz, co ma działać, a nie jak to uruchomić

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Jeden plik, jedna komenda, cała aplikacja

```
compose.yaml

services:
  db:
    image: …
  api:
    build: ./api
  web:
    build: ./web
volumes:
  pgdata:
```



build albo pull dla każdej usługi, potem sieć, wolumeny i kontenery

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

compose.yaml: usługi, sieci, wolumeny

```
services:
  web:
    build: ./app
    ports: ["5001:5000"]
    environment:
      DATABASE_URL: postgresql://app:changeme@db/shop
    depends_on:
      db:
        condition: service_healthy
  db:
    image: postgres:18
    volumes: ["pgdata:/var/lib/postgresql"]
volumes:
  pgdata:
```

`bez version:`
pole przestarzałe, Compose v2 tylko ostrzega

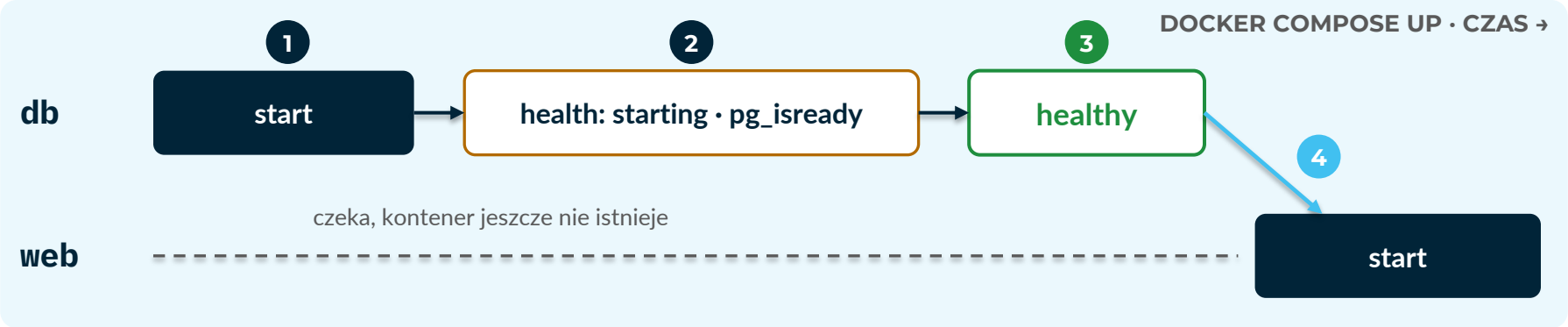
nazwa usługi = DNS
web łączy się z db po nazwie

sieć projektu
<katalog>_default tworzy się sama

build albo image
własny Dockerfile albo gotowy obraz

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

depends_on czeka na healthy, nie na start




```
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U postgres"]
  interval: 5s
  retries: 5
```

depends_on: [db] bez condition
web startuje, gdy kontener db istnieje, a baza jeszcze nie przyjmuje połączeń

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

Profil włącza usługę tylko na żądanie

```
services:
  api:  …
  web:  …
  mongo:  …
  mongo-express:
    image: mongo-express:1.0.2-20-alpine3.19
    profiles: ["debug"]
    ports: ["8081:8081"]
```

to samo: COMPOSE_PROFILES=debug docker compose up -d

ZWYKŁY START

```
$ docker compose up -d
api  mongo  web
```

Z PROFILEM DEBUG

```
$ docker compose --profile debug up -d
api  mongo  web  mongo-express
```

usługa bez profiles startuje zawsze • z profiles tylko po włączeniu profilu

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

watch: zapisany plik sam trafia do kontenera

zapis w IDE ./backend → **docker compose watch** → sync: kopia do kontenera → proces restartuje

zmiana package.json → **docker compose watch** → rebuild: nowy obraz → nowy kontener

```
api:
  develop:
    watch:
      - action: sync+restart
        path: ./backend
        target: /usr/src/app
      - action: rebuild
        path: ./backend/package.json
```

`$ docker compose watch`

bez watch: każda zmiana = up --build

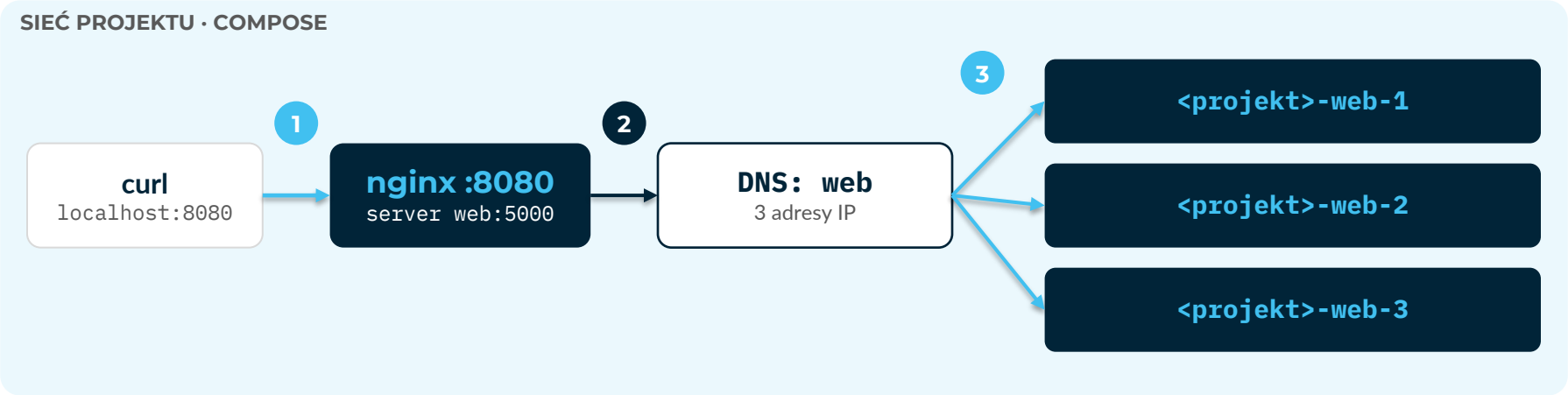
z watch: zapisujesz plik i tyle

tylko dev · do rejestru: zwykły build

sync dla kodu, rebuild dla zależności · watch to narzędzie dev, nie wdrożenie

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

--scale web=3: jedna nazwa, trzy kontenery




```
docker compose up -d --scale web=3
```

```
resolver 127.0.0.11 valid=5s;
server web:5000 resolve;
```

łącz się z nazwą usługi, nie z kontenerem · jutro to samo robi Service

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

▶ DEMO · Compose: up, healthy, scale

Jedno up, kolejność startu i trzy repliki

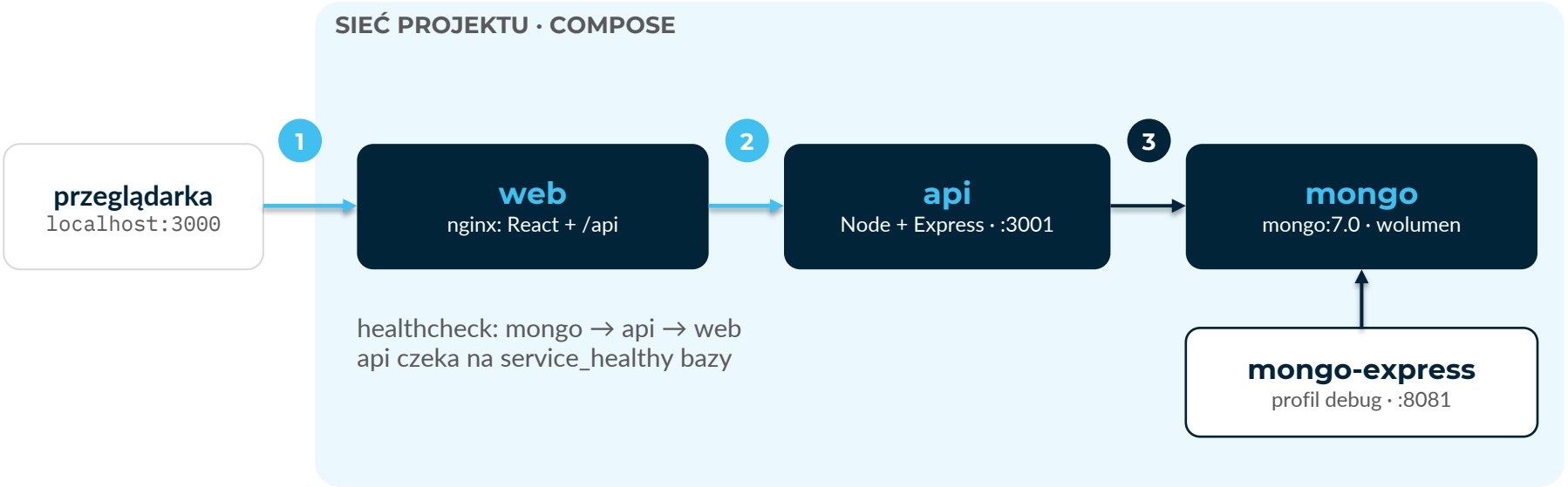
Cel: start po healthcheck, dane po down, potem trzy kopie web za nginx.

OBSERWUJCIE

1. kolejność w logach up
2. health w compose ps
3. który kontener odpowiada

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

ToDos: trzy warstwy, cztery kontenery



jutro wieczorem te same obrazy web i api wdrożysz na Kubernetesa

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

LAB `labs/lab04-compose-todos/`

ToDos w Compose: złóż, zabezpiecz, wypchnij

1. Uruchom plik startowy: docker compose up -d --build
2. Wolumen dla mongo, healthcheck i service_healthy
3. Profil debug z mongo-express, USER i skan obrazów
4. Tag 1.0.0 i push todos-web, todos-api · chętni: watch

SUKCES zadanie dodane w przeglądarce przeżywa down i up, a obrazy web i api z tagiem 1.0.0 są w rejestrze

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Dzień 1 w czterech zdaniach

1. **Kontener to jeden program i żyje tyle, co on**
   CLI → dockerd → containerd → shim → runc · port, zmienne i wolumen przy run
2. **Sieć użytkownika daje nazwy, wolumen trwałość**
   DNS po nazwie usługi, wolumen vs bind vs tmpfs, logi na stdout z rotacją
3. **Obraz to warstwy, cache zależy od kolejności**
   multi-stage, .dockerignore, sekrety przez --mount, tag z wersją, --platform
4. **Minimum uprawnień, a całość w compose.yaml**
   USER, --read-only, nigdy docker.sock · healthcheck, profile, watch, --scale

obrazy ToDos z rejestru: jutro wieczorem ToDos startuje na klastrze

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.

sages

Co było dziś najtrudniejsze?

Jutro · Dzień 2: Kubernetes

architektura i kubectl, Pod i Namespace, Deployment i konfiguracja, Service, Ingress, Gateway API

Copyright (C) 2026 Sages sp. z o.o. Kopiowanie, przetwarzanie, rozpowszechnianie tych materiałów w całości lub w części jest zabronione.